Meaning Under a Class: Hierarchy-Filtered Vector Search Inside an Ontology Store

Lam Nguyen
Case Western Reserve University
Cleveland, Ohio, USA
ltn18@case.edu

ABSTRACT

Ontologies pair a class hierarchy with names that embeddings can compare, and a natural question combines the two: which terms are closest in meaning to a given term *and lie under a given class*? Vector databases answer it by passing the index a filter, and approximate indexes silently lose recall when the filter is selective: on four ontologies, FAISS indexes asked for the ten nearest terms under a class return on average 61% (IVF) and 64% (HNSW) of the right answers, and 2% (HNSW) for a query that lies outside a small class; a 16 times larger beam raises that to only 20%. This paper presents a vector search that lives inside an ontology store and uses the hierarchy as the filter. (1) We prove that no linear order of points keeps nearest neighbors close, and measure how few a sorted index of an ontology puts within a short scan. (2) We label the class hierarchy with postorder intervals in the store's own B+ tree, so that a class's members are a few range scans and membership is a binary search, and we store IVF posting lists as key ranges of the same tree. (3) We show that in an ontology the class filter is correlated with the vectors, so the independence assumption behind filtered-search costing fails, and give four filtering strategies, a cost model for the crossover between resolving the class and testing it per row, and a rule that picks between them. (4) We evaluate against FAISS, hnswlib, and DuckDB on the Gene Ontology, the Human Phenotype Ontology, Uberon, and ChEBI (218,723 terms), with queries that are stored terms and queries that are not. Across 958 queries our rule returns recall 0.999 and never falls below 0.99, with a median of 26 μs; for classes of at most ten members it is 33 times faster than an exact FAISS scan with an ID selector (2.1 against 70 μs).

KEYWORDS

ontology, vector search, filtered nearest neighbor search, class hierarchy, B+ tree

PVLDB Reference Format:
Lam Nguyen. Meaning Under a Class: Hierarchy-Filtered Vector Search Inside an Ontology Store. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The source code, data, and/or other artifacts have been made available at https://github.com/lamng3/OntoDB.

1 INTRODUCTION

An ontology is a class hierarchy whose classes have names, and names can be embedded: the Gene Ontology, the Human Phenotype Ontology, and ChEBI each give tens or hundreds of thousands of terms a vector, so that terms close in meaning are close in cosine similarity. Tasks such as ontology matching, term normalization, and retrieval over annotated data then ask for *the terms nearest to a given term that lie under a given class*: the nearest phenotype to "short stature" under "abnormality of the skeletal system", the nearest process to "apoptotic process" under "cell death". The hierarchy defines the filter, and the vectors rank what passes it.

Vector databases answer such queries by giving the index a predicate, and they leave four problems open. **C1. A sorted index does not hold meaning.** Triple stores answer patterns with range scans over sorted permutations, and it is tempting to believe that terms close in meaning are close in the order, so that a scan could serve similarity. No linear order can keep all neighbors close (Section 3), and on four ontologies the best order we tried finds 38% to 56% of a term's ten nearest neighbors within 100 positions. **C2. Filtered approximate search loses recall without warning.** An IVF or graph index with a filter probes the lists or the walk nearest the query; when few of the terms there pass the filter, it returns fewer, worse answers and reports no error. On our workload FAISS indexes with an ID selector return recall 0.61 (IVF) and 0.64 (HNSW), and 0.02 for HNSW when the query lies outside a small class. **C3. The filter is correlated with the vectors.** Filtered search is costed by assuming the filter is independent of the similarity, so that k/f candidates suffice for a filter that passes a fraction f. In an ontology the neighbors of a term lie under the term's own ancestors, so the estimate is wrong in both directions (Section 4). **C4. The filter lives in another system.** A vector database knows nothing of the hierarchy: the caller must resolve a class to its members, keep a table of them, and keep it current.

Example 1.1. Let q be *apoptotic process* and c the class *cell death* in the Gene Ontology. A vector index returns neighbors from anywhere in the ontology, and a post-filter keeps those under c only if enough of them are. For a query from outside a class of at most ten members, an ID-selector IVF returns on average 5% of the right answers. Here the engine finds the 97 members of c with one range scan of the B+ tree that holds the triples, scores them exactly, and returns *apoptotic process involved in development* (cosine 0.865) first, in under a millisecond in a web page.

Contributions. We build the filter into the store.

Table 1: Notation.

Symbol	Meaning
$H = (\mathcal{C}, E)$	the class hierarchy, a DAG of classes and subclass edges
$\text{desc}(c)$	the classes at or below c, including c
$po(c)$, $lo(c)$	postorder number of c; smallest number in its subtree of the spanning forest
$I(c)$	the set of intervals that covers the numbers of $\text{desc}(c)$
$f(c)$	fraction of terms below c: $\lvert\text{desc}(c)\rvert/N$
N, d	number of terms with vectors; vector dimension
$\mathbf{v}(t)$, sim	unit vector of term t; cosine similarity
q, k	the query vector; the number of neighbors asked for
$\text{near}(q, c, k)$	the k terms of $\text{desc}(c)$ nearest to q
v, ρ	IVF: number of lists; lists probed per query
M, ef	HNSW: links per node; search beam width
G, D	a graph and its diameter
γ_m, γ_h	cost per member of resolving a class; cost of a masked walk
θ	the auto rule's threshold on $\lvert\text{desc}(c)\rvert$

(1) **A bound and a measurement on order** (Section 3): every linear order of a graph's vertices separates some edge by $(N-1)/D$ positions, and on four ontologies no order of the terms holds more than 56% of the ten nearest neighbors within 100 positions.
(2) **The hierarchy as a membership test** (Section 4): postorder intervals in the engine's B+ tree make a class's members a few range scans and membership a binary search, which lets a vector search test the filter per row without resolving it.
(3) **A filtering rule that does not lose recall** (Section 4): four strategies, a measurement that the class filter is correlated with the vectors, a cost model for the crossover between resolving the class and testing it, and a rule that picks between them and is exact for the classes where approximate indexes fail.
(4) **Vector indexes whose lists live in the triple tree** (Section 5): IVF posting lists stored as key ranges of the same B+ tree, so that they page, cache, and latch with the triples.

We evaluate against FAISS, hnswlib, and DuckDB on four ontologies, with queries that are stored terms and queries that are not (Section 6). The code, data scripts, and benchmark programs are at the artifact URL above.

2 PRELIMINARIES

Terms, classes, vectors. An ontology has classes $\mathcal{C}$ related by `rdfs:subClassOf` edges E, forming a directed graph H in which cycles are collapsed (classes on a cycle are equivalent). We write $\text{desc}(c)$ for the classes at or below c, including c. Each of N terms, here the classes of the ontology, has a name, and an embedding model maps the name to a unit vector $\mathbf{v}(t) \in \mathbb{R}^d$; the similarity of two terms is the cosine $\mathbf{v}(t) \cdot \mathbf{v}(u)$.

Definition 2.1 (Hierarchy-filtered nearest neighbors). For a query vector q, a class c, and $k \geq 1$, $\text{near}(q, c, k)$ is the set of k terms $t \in \text{desc}(c)$ with the largest $q \cdot \mathbf{v}(t)$.

Table 2: Share of a term's ten nearest neighbors (cosine) found within ±100 positions, and the median position gap to the nearest neighbor, by order of the terms. 2,000 queries.

		share within ±100				median gap		
Ontology	terms	hierarchy	first-seen id	alphabetical	random	hierarchy	id	random
GO	38,092	**0.38**	0.21	0.33	0.005	45	1,739	10,823
HPO	19,894	**0.56**	0.23	0.28	0.009	6	2,033	5,589
Uberon	15,047	**0.39**	0.23	0.35	0.013	41	930	4,374
ChEBI	218,723	0.23	0.13	**0.41**	0.001	444	4,351	61,743

The query vector is the embedding of a stored term, as in ontology matching between two parts of one ontology, or of text that is not a term, such as a synonym or a word from a document. A term used as the query is never its own neighbor. An answer A is judged by its *recall*, $|A \cap \text{near}(q, c, k)|/k$, against exact search over $\text{desc}(c)$. A system that returns fewer than k answers has a recall below one.

The store. The vectors live in an ontology store, OntoDB [19], that keeps triples in three B+ trees behind a buffer pool and keeps the hierarchy as labels in the same engine. This paper uses three of its parts: the B+ tree and its range scan, the postorder labelling of the hierarchy (Section 4), and the buffer pool that pages all of it.

3 ORDER IS NOT SIMILARITY

A triple store answers a bound prefix with one range scan over a sorted index, and it is tempting to hope that terms close in meaning are close in the order, so that a scan of a window could serve a similarity query.

Lemma 3.1 (Bandwidth). *Let G be a graph with N vertices and diameter D. In every linear order of the vertices some edge joins vertices at least $(N-1)/D$ positions apart.*

The first and last vertices of an order are $N-1$ apart and joined by a path of at most D edges, so some edge of the path covers at least $(N-1)/D$ positions (Appendix A). For the neighbor graph of a d-dimensional grid of $N = g^d$ points (diameter $d(g-1)$) that is at least g^{d-1}/d positions in any order, so a scan of width w cannot hold all neighbors unless w is that large. Space-filling curves keep neighbors close on average and need several scans to cover a query ball; they do not escape the lemma.

What real orders give. We embedded the term names of four ontologies (Section 6) and, for 2,000 random terms, took the ten nearest by cosine. Table 2 gives the share of those neighbors found within ±100 positions of the term in each order: the hierarchy's postorder, the dictionary's first-seen ids (an engine's default order), the alphabetical order of the names, and a random order.
Three things follow. First, no order is enough: the best finds 38% to 56% of the neighbors in a window of 200 positions, a window that is 0.1% to 1.3% of the ontology. Second, which order is best depends on the ontology: the hierarchy's postorder wins on three, and alphabetical order wins on ChEBI, whose names share prefixes ("*N*-acetyl-…"); what the hierarchy supplies is *structure*, not similarity. Third, first-seen ids carry some locality (0.13 to 0.23) that comes from the file the ontology was loaded from and not from the engine. The order is therefore useful for the hierarchy and is no substitute for a vector index.

Algorithm 1: NEAR(q, c, k): the auto rule

```
if c is not a class of H then return ∅;
if |desc(c)| ≤ θ then
    R ← the rows of the keys in the range scans
      [(a, 0, 0), (b+1, 0, 0)) for each [a, b] ∈ I(c);
    return the k rows of R with the largest q · v ; // pre-filter:
      exact
else
    return MaskedHnsw(q, k, ef, r ↦ po(r) ∈ I(c));
```

4 FILTERING BY A CLASS

The hierarchy as a membership test. Collapse each cycle of H into one class, pick for every class with several parents the parent with the longest path from a root, and number the classes in postorder of the resulting spanning forest. In a forest the classes below c are those whose number lies in one interval $[lo(c), po(c)]$; in a DAG each class carries a *set* of intervals $I(c)$, built children first by merging its own interval with the sets of its children (Agrawal et al. [1]), and $|I(c)| \leq 1 + |E'(c)|$ for the $|E'(c)|$ non-tree edges below c. The numbers are keys of the engine's B+ tree, $(po(c), c, 0)$ for each class, so

- the *members* of c are one range scan per interval of $I(c)$, in $O(|I(c)|)$ descents plus the members, and
- whether a class u lies below c is a binary search of $po(u)$ in $I(c)$, with no scan and no memory beyond the intervals.

On the four ontologies the mean class needs 1.3 to 2.0 intervals (at most 4,329, within the bound), against 5.3 million ancestor pairs for ChEBI's closure and 402,305 intervals. The labels are built in 4 to 69 ms for 13,850 to 218,723 classes and rebuilt when the hierarchy changes; term ids are untouched.

Four strategies. A query near(q, c, k) can be answered in four ways, all returning terms of desc(c) only.

Pre-filter resolves c to its members by range scans and scores them exactly: about γ_m |desc(c)| time, where γ_m covers a scan step, a lookup from term to row, and one dot product.

Post-filter asks an index for $k' > k$ answers, keeps those below c (a binary search each), and multiplies k' by four until k remain or the index is exhausted.

Masked search hands the index the predicate "the row's number lies in an interval of $I(c)$", so a list scan or a graph walk skips rows outside c and nothing is resolved in advance; each row touched costs one binary search. Masked IVF keeps probing lists, nearest first, until it holds k allowed rows or has probed all (Algorithm 3); masked HNSW still crosses rows it may not return, to keep the graph connected, and keeps a beam of *ef* allowed candidates.

Auto pre-filters when |desc(c)| $\leq \theta$ and otherwise runs masked HNSW.

LEMMA 4.1 (EXACTNESS AND COMPLETENESS). *Pre-filter returns exactly* near(q, c, k). *Masked IVF returns* min$(k, |\text{desc}(c)|)$ *answers, and exactly* near(q, c, k) *when* $\rho = \nu$.

Pre-filter scores every member; masked IVF stops only with k allowed rows or after all lists, and with all lists probed it has scored every allowed row (Appendix A). Masked HNSW (Algorithm 2) crosses disallowed nodes to keep the graph connected and keeps

Algorithm 2: MASKEDHNSW$(q, k, ef, allowed)$

```
descend the upper layers greedily from the entry point to the node
  nearest q (the filter is not applied);
frontier ← {that node}; W ← ∅ ;      // W holds allowed rows
  only
while frontier ≠ ∅ do
    x ← the frontier node nearest q;
    if |W| ≥ ef and x is farther than the worst row of W then
      break;
    for each unvisited neighbor y of x on layer 0 do
        add y to frontier whether or not it is allowed;
        if allowed(y) then add y to W; drop the worst row if
          |W| > ef;
return the k best rows of W;
```

Algorithm 3: Masked IVF: the k best rows allowed by the class

```
sort the ν lists by the similarity of their centroids to q;
best ← ∅; seen ← 0;
for each list ℓ in that order do
    for each key (ℓ, r, 0) of the prefix scan of ℓ in the B+ tree do
        if the number of row r lies in an interval of I(c) then score
          r; keep it among the k best;
    seen ← seen + 1;
    if seen ≥ ρ and |best| = k then break;
return best;
```

a beam of *ef allowed* rows, so it does not stop early for lack of allowed neighbors, which is also why it is slow for a small class: it walks until it has found *ef* of them. An index asked for k answers and filtered afterwards has neither property.

The filter is correlated with the vectors. Costing a filtered search by the filter's selectivity assumes independence: a filter that passes a fraction $f(c)$ of the terms needs about $k/f(c)$ candidates. In an ontology the names of a class's descendants resemble the class and one another, so the neighbors of a term concentrate under the term's own ancestors. Table 3 measures it: for a query that is a member of c, or a synonym of one, the share of the unfiltered top 100 under c is 0.03 to 0.59, which is 10 to 100 times the fraction of terms below c for classes of up to 1,000 members, and for a query outside it falls to 0.00 to 0.24, below that fraction for the largest classes. The estimate $k/f(c)$ is therefore too large for an inside query, which a post-filter answers cheaply, and too small for an outside one, for which a post-filter's candidates run out before k allowed rows are seen.

Resolve or test. Masked search can test the filter per row (one binary search over $I(c)$) or resolve it first into a bitmap by range scans. Resolving costs $\gamma_m|\text{desc}(c)|$ whether or not the search ever looks at those rows; testing costs only for rows the walk touches. Pre-filtering resolves and scores, so it is the right plan exactly when $\gamma_m|\text{desc}(c)|$ is less than the masked walk's cost γ_h:

$$\text{pre-filter wins} \iff |\text{desc}(c)| < \theta = \gamma_h/\gamma_m.$$

The walk's cost depends on where the query is: for a query inside c the walk finds allowed rows at once, and for one outside it must

Table 3: Share of a query's unfiltered top 100 neighbors that lie under the class, by query kind and number of classes below c (pooled mean; the independence estimate would be the fraction of terms below c).

Below c	independent	inside	outside	synonym
≤10	0.000	0.03	0.00	0.03
11–100	0.001	0.14	0.00	0.11
101–1,000	0.012	0.31	0.01	0.29
1,001–5,000	0.054	0.54	0.03	0.43
>5,000	0.334	0.59	0.24	0.46

cross rows it cannot return, so γ_h and θ differ, and a single threshold has to sit between them (Section 6).

5 VECTOR INDEXES IN THE STORE

The engine keeps one unit vector per term in memory, loaded from a checksummed file whose records name terms by spelling, so a file survives a dictionary that numbers terms differently. Embeddings come from outside; the engine stores and searches them with four index families, each a different trade between exactness, memory, and build time.

Flat. Score every vector with eight independent accumulators (a plain loop is not vectorized by the compiler without fast-math and made our first exact baseline four times too slow): $O(Nd)$ and exact.

IVF with lists in the triple tree. Spherical k-means with k-means++ seeding [3] gives $\nu = 2.6\sqrt{N}$ centroids, each vector goes to its nearest, and a query scores the centroids and reads the ρ nearest lists, for a cost $\nu d + \rho(N/\nu)d$. The design choice is where the lists live: as key ranges of the B+ tree that holds the triples. The pair (list, row) is a key $(\text{list}, \text{row}, 0)$, so a list is a prefix scan, and the lists page, cache, and latch like every other index of the engine, with the buffer pool, the page latches, and the checksums already there; a separate vector library would add its own memory and its own copy of the data. The price is a 24-byte key per row and a random read of the vector per entry, which Section 6 measures against a contiguous list.

HNSW. The layered proximity graph of Malkov and Yashunin [15] with the paper's neighbor-selection heuristic, M=16 links (32 on layer 0), and construction beam 200; search takes an optional row filter and still crosses rows it may not return.

In the engine. The indexes are driven from the shell and from the browser build. After loading the ontology and its vectors and building the labels and the indexes,

```
\hindex build
\vload go.ovec
\vindex build all
\similar "apoptotic process" 4 under "cell death"
```

prints the neighbors with their cosine, the strategy the rule chose, and the time (here for an 834-class slice of the Gene Ontology in a browser):

```
0.865  apoptotic process involved in development
0.802  apoptotic process involved in morphogenesis
0.786  epithelial cell apoptotic process
0.779  neuron apoptotic process     [pre-filter, 0.8 ms]
```

Forcing a strategy (`using post-ivf`, `using mask-hnsw`) runs the others.

Product quantization. Cutting a vector into m pieces and replacing each by its nearest of 256 learned centers [11] gives m bytes per vector, scored through m tables of 256 dot products; it is for memory, not speed, at this size, and we do not use it in the filtered experiments.

6 EXPERIMENTAL STUDY

We evaluate the filtered search along four research questions.

- **[RQ1] Order**: how much of a term's neighborhood does a sorted order of an ontology hold? (Table 2, Section 3.)
- **[RQ2] Filtered search**: what do the strategies return and cost, against FAISS filtering with ID selectors?
- **[RQ3] The rule**: is the class filter correlated with the vectors, does testing it per row beat resolving it, and where is the crossover?
- **[RQ4] The indexes**: what do IVF with lists in the triple tree and HNSW cost against FAISS and hnswlib, and does the design scale to 218,723 terms?

Setup. The ontologies are the Gene Ontology [4] (38,092 terms, release 2026-07-26 without obsolete terms), the Human Phenotype Ontology [13] (19,894), Uberon [17] (15,047), and ChEBI [9] (218,723), each with the `is_a` edges as the hierarchy and the term names embedded by `all-MiniLM-L6-v2` [24] into 384 dimensions on a CPU. For each ontology and each of five sizes of $|\text{desc}(c)|$ (≤10, 11–100, 101–1,000, 1,001–5,000, >5,000) we draw up to 20 classes that have subclasses, and for each class three queries: a stored term inside the class, one outside it, and the embedding of a synonym of a stored member (5,000 synonyms sampled per ontology), which is not a stored vector. Answers are scored against exact search over the class's members, with the query term excluded; there are 958 queries per strategy. OntoDB uses IVF with $\nu = 2.6\sqrt{N}$ lists and $\rho = 16$, HNSW with M=16, construction beam 200, and ef=64, and the auto rule with $\theta = 4{,}096$; FAISS 1.15.1 uses `IndexFlatIP`, `IndexIVFFlat` (same ν, nprobe 16), and `IndexHNSWFlat` (same M, efSearch 64), each searched with an `IDSelectorBitmap` of the class's members; hnswlib 0.8.0 appears in the unfiltered comparison. The members of a class for FAISS come from a descendant table built outside the timer, which favors FAISS. Every time is the mean of three runs after one warm-up, on one thread of an Apple M2 Pro with 16 GiB; FAISS is called once per query from Python, and its call overhead (about 10 μs) is included.

6.1 RQ2: Filtered Search

Approximate indexes lose recall silently. Table 4 and Figure 1 give the result. Over the 958 queries FAISS's IVF and HNSW with an ID selector return recall 0.61 and 0.64, and 59% and 52% of their queries are below 0.99. The loss is worst where the filter is most selective and the query is far from it: for a query outside a class of at most ten members they return 0.05 and 0.02. A larger budget does not repair it (Table 5): with half of all lists probed, IVF reaches 0.52 for those queries at three times the time, and HNSW with a beam of 1,024 reaches 0.20 at 26 times the time, because the walk

Table 4: Ten nearest terms under a class: recall@10 / microseconds per query, by number of classes below c and by query kind, pooled over 4 ontologies. FAISS runs each query with an ID selector of the class's members (their lookup is not timed). Bold marks the fastest strategy with recall at least 0.99; blue marks OntoDB.

Below c	query	DuckDB join	FAISS exact	FAISS IVF	FAISS HNSW	pre-filter	masked HNSW	auto
≤10	inside c	1.00 / 718	1.00 / 71	0.79 / 29	0.87 / 50	**1.00 / 2.1**	1.00 / 42.1 k	1.00 / 2.2
11–100		1.00 / 811	1.00 / 72	0.82 / 30	0.85 / 49	**1.00 / 7.9**	1.00 / 37.4 k	1.00 / 8.1
101–1,000		1.00 / 1.2 k	1.00 / 90	0.89 / 35	0.91 / 50	**1.00 / 47**	1.00 / 5.9 k	1.00 / 49
1,001–5,000		1.00 / 2.6 k	**1.00 / 213**	0.93 / 46	0.98 / 47	1.00 / 244	1.00 / 1.2 k	1.00 / 396
>5,000		1.00 / 28.6 k	1.00 / 2.0 k	0.93 / 81	0.98 / 56	1.00 / 4.9 k	1.00 / 506	**1.00 / 456**
≤10	outside c	1.00 / 708	1.00 / 69	0.05 / 29	0.02 / 51	**1.00 / 2.1**	1.00 / 42.2 k	1.00 / 2.2
11–100		1.00 / 802	1.00 / 71	0.06 / 29	0.04 / 49	**1.00 / 7.8**	1.00 / 38.8 k	1.00 / 8.3
101–1,000		1.00 / 1.1 k	1.00 / 88	0.23 / 29	0.24 / 50	**1.00 / 47**	1.00 / 18.2 k	1.00 / 51
1,001–5,000		1.00 / 2.6 k	**1.00 / 205**	0.40 / 33	0.39 / 49	1.00 / 245	1.00 / 9.3 k	1.00 / 652
>5,000		1.00 / 27.4 k	1.00 / 2.0 k	0.58 / 61	0.73 / 62	1.00 / 5.0 k	0.99 / 1.9 k	**0.99 / 1.9 k**
≤10	synonym of a member	1.00 / 708	1.00 / 49	0.74 / 25	0.76 / 48	**1.00 / 2.2**	1.00 / 25.3 k	1.00 / 2.3
11–100		1.00 / 808	1.00 / 56	0.74 / 27	0.74 / 47	**1.00 / 7.7**	1.00 / 27.1 k	1.00 / 8.0
101–1,000		1.00 / 1.1 k	1.00 / 87	0.82 / 33	0.88 / 51	**1.00 / 47**	1.00 / 8.9 k	1.00 / 50
1,001–5,000		1.00 / 2.6 k	**1.00 / 203**	0.79 / 42	0.90 / 50	1.00 / 246	1.00 / 3.7 k	1.00 / 257
>5,000		1.00 / 28.1 k	**1.00 / 2.0 k**	0.88 / 75	0.91 / 59	1.00 / 5.1 k	0.98 / 615	0.98 / 596

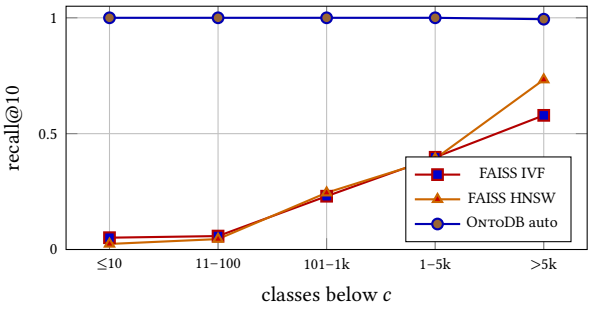


Figure 1: Recall@10 for queries outside the class, by number of classes below it (pooled over the four ontologies).

Table 5: FAISS with an ID selector at its default budget and at a much larger one (half of all IVF lists; HNSW beam 1,024): recall@10 / microseconds, queries outside the class, pooled.

Below c	IVF	IVF, half the lists	HNSW	HNSW, beam 1,024
≤10	0.05 / 29	0.52 / 86	0.02 / 51	0.20 / 1.3 k
11–100	0.06 / 29	0.68 / 88	0.04 / 49	0.35 / 1.2 k
101–1,000	0.23 / 29	0.89 / 99	0.24 / 50	0.66 / 1.2 k
1,001–5,000	0.40 / 33	0.97 / 160	0.39 / 49	0.78 / 1.2 k
>5,000	0.58 / 61	0.99 / 1.1 k	0.73 / 62	0.95 / 1.5 k

and the probed lists are the ones near the query, where no member lives. Over all queries the wide IVF reaches recall 0.91 in 204 μs on average, the same time as the rule, and 19% of its queries are still below 0.99; the wide HNSW reaches 0.81 in 1.2 ms.

Why. Table 6 shows the mechanism for IVF. The lists an IVF index probes are the ones nearest the query, and the members of a small class are somewhere else: for a query outside a class of at most ten members (four on average), the 16 probed lists hold 0.2 of them, and outside a class of 11 to 100 (31 on average) 1.1, where the answer needs ten. For a query inside, they hold 3.5 of four and 19 of 31. An index that cannot find allowed rows in the region it searches

Table 6: Why an ID-selector IVF loses recall: the members of the class that lie in the 16 lists it probes, against the ten the answer needs (mean over the queries, pooled over the four ontologies).

Below c	members	in probed lists, inside	outside
≤10	4	3.5	0.2
11–100	31	19.2	1.1
101–1,000	308	120.8	12.6
1,001–5,000	1,856	397.7	48.1
>5,000	36,431	1,139.9	585.4

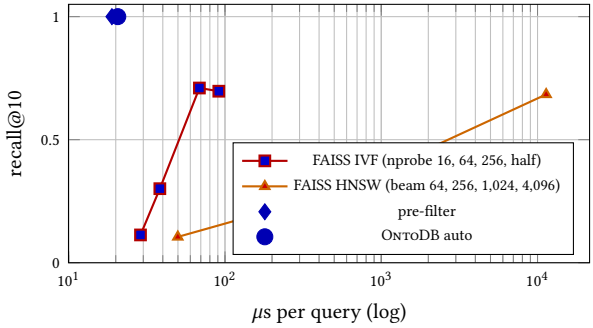


Figure 2: What more search effort buys FAISS on queries outside classes of up to 1,000 members: recall@10 against mean microseconds, pooled over the ontologies, for four IVF and four HNSW budgets, and the two strategies of the rule.

returns what it has, and a filtered HNSW walk behaves the same way: it fills its beam with near nodes that are not members.

A relational baseline. DuckDB with a closure table of (class, member) pairs and one SQL statement per query, which joins the class's members to their vectors and orders by inner product, is exact like pre-filtering and has nothing to lose recall: its mean is 4.0 ms and its median 0.97 ms, 0.7 ms for classes of at most ten members, where the rule takes 2.1 μs, and 28 ms for classes above

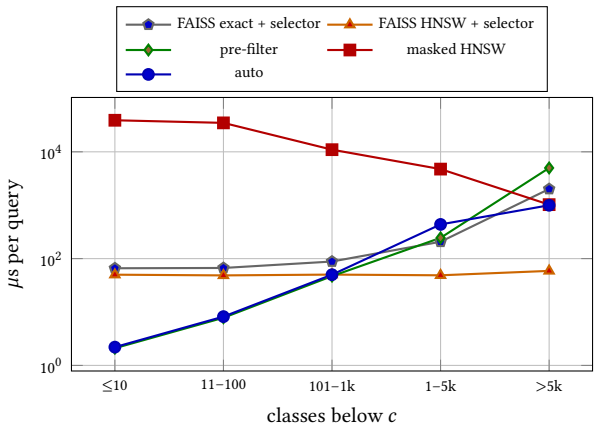


Figure 3: Mean microseconds per query by number of classes below c, pooled over the ontologies and query kinds (log scale).

5,000. The hierarchy is the same in both; the difference is that a range scan of a B+ tree that already holds the labels replaces a join, and a dot-product loop replaces a query plan.

Interval filtering is exact and fast where it matters. The auto rule returns recall 0.999, and none of its 958 queries is below 0.99; its median is 26 μs and its mean 206 μs. An exact FAISS scan with the same selector has median 60 μs and mean 300 μs, so the rule is 2.3 times faster at the median and 1.5 times on average, while the approximate indexes, which are 4 to 6 times faster than the rule on average (50 and 36 μs), are wrong more often than right. The gain is largest for small classes: with at most ten members, pre-filtering takes 2.1 μs, 33 times less than the 70 μs of the FAISS scan, which must test a bitmap against all N rows. From 1,001 to 5,000 members the FAISS scan is faster than pre-filtering (214 against 244 μs inside) and than the rule (396 μs inside, 652 outside); above 5,000 members the rule's masked HNSW beats the exact scan for queries inside the class (456 against 2,000 μs) and matches it for queries outside (1.9 against 2.0 ms). Synonym queries, whose vectors are not stored, behave like inside queries.

6.2 RQ3: The Rule

Table 3 shows the correlation: for a query inside a class of 101 to 1,000 members, 31% of the unfiltered top 100 lie under the class, where an independent filter would pass 1.2% of the terms, and for a query outside it 1%; for the largest classes 59% inside against 33% expected, and 24% outside. The independence estimate $k/f(c)$ of the candidates a post-filter needs is therefore too high inside and too low outside, and the post-filtering strategies show it: post-filtered HNSW takes 34.5 ms on average because it must widen until it finds k allowed rows, and post-filtered IVF has recall 0.61 because it stops widening too soon.

Testing the filter per row costs the same as resolving it into a bitmap up to 5,000 members (Table 7) and 2.7 times less above, where resolving takes more time than the walk it serves. The cost model of Section 4 predicts the crossover: the mean pre-filter cost per member is $\gamma_m = 0.15\,\mu$s (median 0.11), and the masked walk over classes of more than 5,000 members takes $\gamma_h = 506\,\mu$s for

Table 7: Masked HNSW with the filter tested per row against the class's intervals, and with the filter resolved first into a bitmap by range scans (resolving is timed): microseconds per query, pooled over the ontologies.

Below c	interval test	resolve, then bitmap	ratio
≤10	42.2 k	41.1 k	1.0×
11–100	38.1 k	37.4 k	1.0×
101–1,000	12.0 k	11.9 k	1.0×
1,001–5,000	5.3 k	5.4 k	1.0×
>5,000	1.2 k	3.3 k	2.7×

Table 8: The auto rule at other thresholds: mean microseconds per query over every class size and query kind (all strategies return recall at least 0.99 except as noted).

Threshold	GO	HPO	Uberon	ChEBI
256	609	246	301	6.9 k
1,024	456	169	71	3.2 k
4,096	**111**	**59**	**41**	533
16,384	148	62	79	**426**
65,536	180	114	79	567

Table 9: A second embedding model, `all-mpnet-base-v2` (768 dimensions): recall@10 / mean microseconds over every class size and query kind, and the share of the unfiltered top 100 under the class for queries inside and outside.

Ontology	FAISS IVF	FAISS HNSW	pre-filter	auto	top-100 under c, inside	outside
GO	0.64 / 51	0.67 / 81	1.00 / 279	1.00 / 195	0.30	0.02
HPO	0.68 / 43	0.71 / 77	1.00 / 181	1.00 / 94	0.37	0.05

Table 10: The number of neighbors asked for, on the Gene Ontology: recall / mean microseconds over every class size and query kind.

k	FAISS IVF	FAISS HNSW	pre-filter	auto
1	0.70 / 31	0.75 / 46	1.00 / 173	1.00 / 107
10	0.63 / 31	0.67 / 47	1.00 / 178	1.00 / 110
50	0.55 / 33	0.56 / 49	1.00 / 182	1.00 / 115

queries inside, 615 for synonyms, and 1,942 for outside queries, so $\theta = \gamma_h/\gamma_m$ is about 3,400 to 13,000.

Table 8 confirms it: a threshold of 4,096 is best on the Gene Ontology, the Human Phenotype Ontology, and Uberon, and 16,384 on ChEBI, whose walks are costlier; 256 and 1,024 are 1.7 to 7 times slower than the best, and 65,536 is 1.3 to 1.9 times slower. A single threshold of 4,096 is within 25% of the best on all four.

The number of neighbors. Table 10 varies k on the Gene Ontology: asking for one neighbor or fifty changes the approximate indexes' recall by at most 0.1 (0.70 to 0.55 for IVF), and the rule stays exact, at a mean of 107 to 115 μs.

Another embedding model. Table 9 repeats the study on the Gene Ontology and the Human Phenotype Ontology with `all-mpnet-base-v2`, a larger model with 768 dimensions, and the picture holds: FAISS's IVF and HNSW return recall 0.64 to 0.71, the rule returns 1.00, and the correlation is as strong (0.30 and 0.37 of

the unfiltered top 100 under the class for inside queries, 0.02 and 0.05 for outside).

Threats to validity. The members FAISS filters with come from a table built outside the timer, which favors it; FAISS and DuckDB are called from Python, whose per-call overhead (about 10 μs) matters for the 2 μs queries; each index is built once, so HNSW's randomness is one draw; the ontologies are four of the OBO Foundry's, with names embedded by two sentence models, and the queries are sampled by class size, not by use. The comparison with FAISS's approximate indexes is one of default-style settings and of a wide budget, not of tuned ones.

6.3 RQ4: The Indexes

Table 11 compares the indexes unfiltered. Called one query at a time, OntoDB's exact scan is 2.0 times slower than FAISS's, its IVF 2.8 to 4.4 times slower, and its HNSW 2.0 to 4.2 times slower on the three smaller ontologies and 0.8 times (faster) on ChEBI; recalls match to within 0.01. The IVF gap is the price of the design: each list entry is a decoded 24-byte key and a random read of the vector, about 2.7 times the cost per scored row of a contiguous list, bought in exchange for lists that page, cache, and latch with the triples and need no second copy of the data. Building is slower too (IVF 115 s and HNSW 142 s on ChEBI against 5 and 83 s). Batching favors FAISS further, since its flat scan becomes a matrix product (Appendix, Table 14). On ChEBI, 218,723 terms and 320 MiB of vectors, the rule still returns recall 0.99 or more on every query, with a mean of 543 μs (FAISS's exact scan: 846 μs).
On eight threads (Table 12) the rule scales 5.0 to 5.9 times on every ontology, because its search shares only the read-only index and the engine's latch-free buffer pool; FAISS's exact scan scales 2.4 to 4.7 times through Python threads. At eight threads the rule serves 1.9, 2.3, and 1.5 times the queries of FAISS's exact scan on HPO, Uberon, and ChEBI, and 0.7 times on the Gene Ontology, where the exact scan is faster at one thread as well.

7 RELATED WORK

Vector search. Approximate nearest neighbor search has inverted files and product quantization [11], proximity graphs such as HNSW [15], and disk-oriented graphs such as DiskANN [10]; FAISS [5, 12] and hnswlib implement them, and vector database systems such as Milvus [25] wrap them with storage and a query language; Pan, Wang, and Li survey the hybrid queries, an attribute predicate plus a vector, that are hard for them [22].

Filtered search. Filtered-DiskANN builds a graph whose edges respect label filters [7], and ACORN makes HNSW traverse only the nodes that pass a predicate and densifies the graph so that it stays connected [23]. Both treat the filter as an opaque attribute of the vector; a class filter is not opaque, because the hierarchy supplies a structure that makes membership a binary search and the filter correlated with the vectors, which is what we measure and use. FAISS's ID selectors, the baseline here, pass a predicate to the index's own search.

Hierarchies in stores. Agrawal, Borgida, and Jagadish [1] label a hierarchy with postorder intervals, and Grust [8] numbers XML trees; RDFS entailment is otherwise rewritten or materialized [16]. Neo4j, Neptune Analytics, Fluree, and GraphDB add vector or similarity indexes to graph and RDF stores [2, 6, 18, 21]; in the documentation we read, none restricts the search with range-scannable hierarchy intervals in the engine that holds the triples. Embeddings of the hierarchy itself, such as hyperbolic embeddings [20], would change the vectors, not the filter.

Order and similarity. Space-filling curves and sorting keys map points to one dimension and keep neighbors close on average; the bandwidth argument says they cannot keep all of them close, and the adaptive radix tree [14] orders keys by byte and falls under the same bound.

8 CONCLUSION

An ontology's hierarchy is a better filter for vector search than an attribute the index cannot see. Postorder intervals in the store's B+ tree make a class's members a few range scans and membership a binary search; a vector search can then test the filter per row, or resolve it and score the members exactly, and a cost model with one threshold picks the cheaper. On four ontologies this answers near(q, c, k) with recall 0.999 and a median of 26 μs, where FAISS indexes with ID selectors return recall 0.61 and 0.64 and lose almost every answer for a query outside a small class, and where an exact FAISS scan is 33 times slower for classes of at most ten members. No sorted order of the terms can do the same: every order separates some neighbors by $(N-1)/D$ positions, and the best we found holds at most 56% of a term's neighbors in a window of 200.

Limitations. Our IVF and HNSW are 2 to 4 times slower than FAISS's when called one query at a time and take longer to build, the price of keeping lists in the triple tree; the vectors are held in memory and the indexes are rebuilt, not updated, when they change; the labels are rebuilt when the hierarchy changes; the embeddings come from one small model that embeds names only; masked HNSW is slow for classes of a few hundred members and under, which the rule avoids and a denser graph, as ACORN builds [23], might fix; and for classes of 1,001 to 5,000 members an exact FAISS scan with a precomputed member bitmap is faster than our rule.

REFERENCES

[1] Rakesh Agrawal, Alexander Borgida, and H. V. Jagadish. 1989. Efficient Management of Transitive Relationships in Large Data and Knowledge Bases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 253–262. https://doi.org/10.1145/67544.66950

[2] Amazon Web Services. 2023. Amazon Neptune Analytics: Vector Similarity Search. AWS documentation and launch announcement. https://aws.amazon.com/blogs/aws/introducing-amazon-neptune-analytics-a-high-performance-graph-analytics, accessed October 2026.

[3] David Arthur and Sergei Vassilvitskii. 2007. k-means++: The Advantages of Careful Seeding. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1027–1035.

[4] Michael Ashburner et al. 2000. Gene Ontology: Tool for the Unification of Biology. *Nature Genetics* 25 (2000), 25–29.

[5] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. *arXiv preprint arXiv:2401.08281* (2024).

[6] Fluree PBC. 2026. Fluree Vector Search. Fluree documentation. https://labs.fluree.ee/docs/db/indexing-and-search/vector-search, accessed October 2026.

[7] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri.

Table 11: Unfiltered ten-nearest-neighbor search, 1,000 queries: recall@10 / microseconds per query. FAISS is called once per query from Python, like OntoDB (its call overhead, about 10 μs, is included); batched numbers are in the appendix. IVF: nprobe 16; HNSW: M=16, ef=40. Build seconds in parentheses.

Ontology	terms	flat (OntoDB)	flat (FAISS)	IVF (OntoDB)	IVF (FAISS)	HNSW (OntoDB)	HNSW (FAISS)
GO	38,092	1.00 / 2.2 k	1.00 / 1.1 k	0.93 / 289 (18 s)	0.93 / 71	1.00 / 123 (16 s)	1.00 / 58
HPO	19,894	1.00 / 1.2 k	1.00 / 610	0.93 / 222 (8 s)	0.92 / 50	1.00 / 190 (9 s)	1.00 / 45
Uberon	15,047	1.00 / 907	1.00 / 432	0.91 / 142 (5 s)	0.91 / 50	1.00 / 77 (6 s)	1.00 / 39
ChEBI	218,723	1.00 / 13.1 k	1.00 / 6.6 k	0.93 / 1.1 k (115 s)	0.92 / 295	0.99 / 169 (142 s)	0.98 / 201

Table 12: Filtered queries per second on one to eight threads over the whole workload: the auto rule (C++ threads, one shared index) against FAISS's exact scan and HNSW with an ID selector (Python threads; FAISS releases the interpreter lock while it searches).

	auto		FAISS exact		FAISS HNSW	
Ontology	1	8	1	8	1	8
GO	6,144	31k	9,106	42k	11k	43k
HPO	15k	88k	17k	46k	15k	40k
Uberon	21k	119k	21k	51k	17k	40k
ChEBI	1,327	7,576	1,085	5,091	7,001	35k

2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference*. 3406–3416.

[8] Torsten Grust. 2002. Accelerating XPath Location Steps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 109–120.

[9] Janna Hastings et al. 2016. ChEBI in 2016: Improved Services and an Expanding Collection of Metabolites. *Nucleic Acids Research* 44, D1 (2016), D1214–D1219.

[10] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[11] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[13] Sebastian Köhler et al. 2021. The Human Phenotype Ontology in 2021. *Nucleic Acids Research* 49, D1 (2021), D1207–D1217.

[14] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 38–49.

[15] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[16] Boris Motik, Yavor Nenov, Robert Piro, Ian Horrocks, and Dan Olteanu. 2014. Parallel Materialisation of Datalog Programs in Centralised, Main-Memory RDF Systems. In *Proceedings of AAAI*.

[17] Christopher J. Mungall, Carlo Torniai, Georgios V. Gkoutos, Suzanna E. Lewis, and Melissa A. Haendel. 2012. Uberon, an Integrative Multi-Species Anatomy Ontology. *Genome Biology* 13, 1 (2012), R5.

[18] Neo4j, Inc. 2026. Neo4j Vector Index and Search. Neo4j documentation. https://neo4j.com/developer/genai-ecosystem/vector-search/, accessed October 2026.

[19] Lam Nguyen. 2026. OntoDB: A Fast and Persistent Query Engine for Ontologies. Preprint; https://github.com/lamng3/OntoDB.

[20] Maximilian Nickel and Douwe Kiela. 2017. Poincaré Embeddings for Learning Hierarchical Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[21] Ontotext. 2019. GraphDB 8.9 with Improved Semantic Similarity Search. Ontotext announcement. https://sirma.com/news/graphdb-8-9-with-improved-semantic-similarity-search.html.

[22] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of Vector Database Management Systems. *The VLDB Journal* 33 (2024), 1591–1615.

[23] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 120.

[24] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.

[25] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 2614–2627.

Table 13: Per ontology, over every class size and query kind: recall@10, and mean / median microseconds per query.

Ontology	queries	auto	FAISS flat	FAISS IVF	FAISS HNSW
GO	247	1.00 / 110 / 26	1.00 / 117 / 46	0.63 / 31 / 27	0.67 / 47 / 47
HPO	237	1.00 / 59 / 18	1.00 / 66 / 29	0.67 / 27 / 23	0.72 / 45 / 46
Uberon	201	1.00 / 41 / 8.3	1.00 / 57 / 22	0.62 / 23 / 20	0.67 / 43 / 43
ChEBI	273	1.00 / 543 / 59	1.00 / 846 / 224	0.55 / 59 / 50	0.54 / 63 / 65

APPENDIX

A PROOFS

Proof of Lemma 3.1. The first and last vertices of the order are $N-1$ apart and joined by a path of at most D edges, so some edge of the path covers at least $(N-1)/D$ positions. □

Proof of Lemma 4.1. Pre-filter scores every member of $\mathrm{desc}(c)$ and keeps the k best, which is $\mathrm{near}(q, c, k)$ by definition. Masked IVF scores only rows whose number lies in an interval of $I(c)$, which are the members; it stops when it holds k of them after ρ lists, or after all ν lists, so it returns k members if $|\mathrm{desc}(c)| \geq k$ and all otherwise. With $\rho = \nu$ it has scored every member and keeps the k best, which is $\mathrm{near}(q, c, k)$. □

B MORE MEASUREMENTS

Table 14: Unfiltered search with FAISS and hnswlib answering the 1,000 queries in one batch on one thread: recall@10 / microseconds per query. The flat scan is a matrix product, which one-query-at-a-time search cannot use.

Ontology	flat	IVF (nprobe 16)	HNSW (*ef*=40), FAISS	HNSW (*ef*=40), hnswlib	IVF (nprobe 64)
GO	1.00 / 30	0.93 / 60	1.00 / 58	1.00 / 250	0.98 / 192
HPO	1.00 / 17	0.92 / 38	1.00 / 44	1.00 / 256	0.97 / 132
Uberon	1.00 / 13	0.91 / 35	1.00 / 36	1.00 / 241	0.97 / 120
ChEBI	1.00 / 162	0.92 / 124	0.98 / 91	0.98 / 299	0.96 / 447